

16-720 Computer Vision

Assignment 1: Local Features and Object-Locked Video

Fall 2026

Released September 8, 2026

This PDF is the authoritative assignment specification. The README explains setup; the notebook supplies executable examples. Read Sections 4–5 before beginning.

Item	Requirement
Release	September 8, 2026
Due	September 29, 2026, 11:59 pm ET (Gradescope)
Base score	100 points
Optional credit	Up to 10 points total, including showcase participation
Expected time	8–10 hours
Runtime	Google Colab; required work runs on CPU
Submit	Python code, completed notebook, LaTeX-generated PDF, and hero-result MP4

2. Overview

In this assignment you build the classical local-feature pipeline behind image matching: Harris corner detection, a compact gradient descriptor, and ratio-test matching. Part A derives the mathematics, Part B implements the pipeline from basic array operations, Part C measures where the pipeline holds up and where it breaks, and Part D puts it to work on video you shoot yourself: a short object-locked video in which a chosen target appears nailed in place while the rest of the scene moves around it.

The robust geometric estimator, warper, smoothing utilities, video input/output, and plotting infrastructure are supplied because their geometry appears later in the course. Your graded Part B detector, descriptor, and matcher must provide the correspondences used by that infrastructure.

3. Learning objectives

After completing this assignment, you should be able to:

- represent local image structure with gradients, vectors, and matrices;
- derive a centered second-moment matrix by eliminating a photometric nuisance parameter;
- interpret second-moment eigenvalues as flat regions, edges, and corners;
- implement deterministic interest-point detection, local description, and feature matching;
- explain which nuisance changes a descriptor does and does not suppress;
- measure and diagnose correspondence failures; and
- integrate student-built visual features with supplied geometric and video components.

4. Academic-integrity and generative-AI policy

The policy is defined by the work being completed, not by the file or chat in which it occurs.

Work	Policy
Part A	Generative AI is not permitted to generate, explain, solve, check, or rewrite responses.
Part B required implementation	Generative AI is not permitted to generate, complete, translate, explain, debug, or rewrite the required functions.
Part C	Generative AI is permitted with disclosure.
Part D	Generative AI is permitted and encouraged with disclosure, but it may not replace or modify the required Part B implementation.
Part E	Generative AI is permitted with disclosure.

For Part A and the required Part B implementation, you may use course notes, textbooks, papers, conventional web resources, official NumPy/OpenCV documentation, and course staff as allowed by the collaboration policy. Black-box feature routines do not satisfy the implementation requirements.

Generative AI is never required. If you do not use it in a permitted section, say so and complete the no-AI alternative in D4. In all cases, you are responsible for understanding, testing, and disclosing what you submit.

Working with others. Talk to your classmates. Discussing the derivations, debugging strategies, and experimental design with other students is one of the best ways to learn this material, and we encourage it — as is coming to office hours early and often. The line is simple: share ideas, not artifacts. Everything you submit — derivations, code, figures, and text — must be produced by you, and you may not read or copy another student’s solutions or let anyone copy yours. List everyone you worked with in your acknowledgments; collaboration is never penalized, but unacknowledged copying is an integrity violation.

5. Shared conventions and allowed tools

- Course image loaders produce two-dimensional NumPy `float32` arrays normalized to `[0, 1]`. Required numerical functions do not clip or renormalize their inputs.
- Image arrays are indexed as `[y, x]`; keypoints are stored in Cartesian `(x, y)` order.
- Keypoint arrays have shape `(N, 2)`. Empty outputs keep their documented rank and width.
- Pixel centers lie at integer coordinates, and positive image `y` points downward.
- The shared random seed is `16720`.
- Required functions must not mutate their inputs.
- Required functions must validate ranks, compatible shapes, and documented parameter ranges. A nonfinite response pixel in NMS and a nonfinite descriptor row in matching are invalid candidates and are discarded as specified below.
- Use basic NumPy array operations and the supplied helpers for the required Part B functions.
- Do not replace any required function with `cv2.cornerHarris`, `cv2.goodFeaturesToTrack`, SIFT, ORB, `BFMatcher`, FLANN, a learned feature system, or an equivalent black box. Such methods may be used only for a clearly separated Part E comparison after the required pipeline is complete.

Every graded figure and table must appear both as a saved output in the executable notebook and in the matching labeled section of the written PDF.

6. Part A — THINK: Local Image Structure (25 points; 1.5–2 hours)

Complete Part A without generative AI. Typeset all work in LaTeX and show intermediate reasoning.

6.1 A1 — From image shifts to the Harris matrix (10 points)

In lecture, Harris was derived from an unweighted shift error. Here you will analyze a photometrically compensated variant. Let $w_i \geq 0$, $\sum_i w_i = 1$, $\mathbf{d} = [u \ v]^\top$, and

$$E(\mathbf{d}, c) = \sum_i w_i [I(\mathbf{x}_i + \mathbf{d}) - I(\mathbf{x}_i) - c]^2,$$

where the unknown scalar c models a small additive brightness change. With $\mathbf{g}_i = \nabla I(\mathbf{x}_i)$, use

$$I(\mathbf{x}_i + \mathbf{d}) \approx I(\mathbf{x}_i) + \mathbf{g}_i^\top \mathbf{d}$$

to do the following:

1. derive the quadratic approximation to $E(\mathbf{d}, c)$ in terms of the gradients;
2. minimize over c and show that $c^*(\mathbf{d}) = \bar{\mathbf{g}}^\top \mathbf{d}$, where $\bar{\mathbf{g}} = \sum_i w_i \mathbf{g}_i$;
3. show that the minimized error is $E(\mathbf{d}, c^*) \approx \mathbf{d}^\top M_c \mathbf{d}$ and derive

$$M_c = \sum_i w_i (\mathbf{g}_i - \bar{\mathbf{g}})(\mathbf{g}_i - \bar{\mathbf{g}})^\top;$$

4. compare M_c with the ordinary matrix $M = \sum_i w_i \mathbf{g}_i \mathbf{g}_i^\top$. Identify the gradient structure removed by compensation and explain when that removal might help or hurt detection.

This problem deliberately adds a nuisance parameter to the lecture derivation; eliminating and interpreting it is the central step.

6.2 A2 — Work through one patch by hand (7 points)

The following 7×7 grayscale patch uses zero-based coordinates, with y increasing downward and x increasing rightward:

$$I = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 \\ 0 & 0 & 1 & 3 & 3 & 3 & 3 \\ 0 & 0 & 1 & 3 & 5 & 5 & 5 \\ 0 & 0 & 1 & 3 & 5 & 7 & 7 \\ 0 & 0 & 1 & 3 & 5 & 7 & 9 \end{bmatrix}.$$

6.2 A2 — continued

At the locations

$$S = \{(3, 3), (4, 3), (3, 4), (4, 4)\},$$

compute

$$I_x(x, y) = \frac{I(x+1, y) - I(x-1, y)}{2}, \quad I_y(x, y) = \frac{I(x, y+1) - I(x, y-1)}{2},$$

and let $\mathbf{g}(x, y) = [I_x(x, y) \ I_y(x, y)]^\top$. For this problem only, use the unnormalized sum

$$M = \sum_{(x,y) \in S} \mathbf{g}(x, y) \mathbf{g}(x, y)^\top.$$

Do **not** divide this sum by four.

1. Show every finite-difference calculation and list the four gradient vectors.
2. Construct M explicitly.
3. Compute both eigenvalues and the corresponding eigenvector directions.
4. Classify the patch as a flat region, edge, or corner.
5. Explain geometrically why the eigenvalue pattern supports the classification.

Do not use a numerical linear-algebra package. For a symmetric matrix $\begin{bmatrix} a & b \\ b & d \end{bmatrix}$, begin with $\det(M - \lambda I) = 0$, then solve a nonzero row of $(M - \lambda I)\mathbf{v} = 0$. The sign of an eigenvector is arbitrary.

6.3 A3 — Predict detector behavior (4 points)

The lecture notes present more than one corner-response function. Throughout this assignment, use the response from the original Harris–Stephens patch — the same one you will implement in B2:

$$R = \det(M) - \kappa \operatorname{tr}(M)^2 = \lambda_1 \lambda_2 - \kappa (\lambda_1 + \lambda_2)^2, \quad \kappa = 0.04.$$

Without running code, predict this response for each case and explain the prediction using gradients, the second-moment matrix, and/or its eigenvalues:

1. a uniform-intensity patch;
2. a strong straight edge;
3. a corner;
4. the same corner after adding a constant brightness offset to every pixel, assuming no clipping or quantization; and
5. the same corner after strong Gaussian blur.

6.4 A4 — Representation and invariance (4 points)

Consider three local-patch representations: raw pixel intensities, image gradients, and spatial histograms of gradient orientation. The third is the descriptor family you will implement in B3; answer from the descriptions given here — no B3 implementation details are required.

1. For raw pixels, state one kind of information they retain and one nuisance change to which they are brittle.
2. For the transition from raw pixels to gradients, identify one nuisance transformation for which gradients should be more robust and one piece of information that is lost.
3. For the transition from gradients to spatial orientation histograms, identify one nuisance transformation for which histograms should be more robust and one piece of information that is lost.
4. State whether the released descriptor has intrinsic rotation or scale invariance and justify the answer from its construction.

7. Part B — BUILD: Detect, Describe, Match (35 points; about 3 hours)

Part B uses the ordinary, uncentered second-moment matrix M ; do not substitute the compensated M_c from A1.

The required implementations in B2–B4 must be your own and must be completed without generative AI. Supplied image loading, Gaussian filtering, plotting, perturbation, geometry, and video functions are infrastructure rather than graded implementations.

Allowed operations in the graded functions. Permitted: plain NumPy array operations — arithmetic, slicing and indexing, broadcasting, `sort/argsort`, `linalg.norm`, `arctan2`, and the like — plus the supplied `gaussian_blur` helper for all Gaussian aggregation. Not permitted: any routine that performs a graded step for you (Section 5 lists the black boxes), or a convolution routine in place of the supplied helper. Validate inputs as each contract specifies and raise `ValueError` for invalid parameters. Degenerate inputs are part of the contract: a flat image, an empty keypoint array, or fewer than two valid destination descriptors must return the documented empty shapes rather than raise. The public tests in `tests/test_pal_public.py` encode these contracts exactly, including tiny and empty inputs — run them early and often.

7.1 B1 — Read the pipeline (5 points)

Run the starter through its convention and supplied-helper cells, inspect the named operations, and answer all five questions in the PDF.

1. A grayscale image has shape `(480, 640)`. What are the shapes of I_x and I_y , and at which array entry is keypoint `[137, 82]` accessed?
2. Inspect the supplied separable Gaussian aggregation helper. Why does its output preserve `(H, W)`, and how are samples beyond the image boundary defined?
3. With `patch_size=16`, `cells=4`, and `bins=8`, state the exact patch bounds around integer keypoint `(x, y)`, the spatial size of each cell, and the descriptor length.
4. What does L2 normalization do to the descriptor under a global positive contrast scaling of the image? What descriptor is produced by a flat patch?
5. The matcher receives reference descriptors as its first argument and current-frame descriptors as its second. Given `matches`, which reference/current keypoint arrays are passed to the supplied estimator, and in which direction does the estimated transform map points?

7.2 B2 — Harris detector (12 points)

Implement these functions in `student/pal.py`:

```
compute_image_gradients(image) -> (Ix, Iy)
compute_second_moment(Ix, Iy, sigma) -> (Sxx, Sxy, Syy)
compute_corner_response(Sxx, Sxy, Syy, k) -> R
nonmax_suppression(R, threshold, radius, max_points) -> (keypoints, scores)
```

Use the following fixed contract.

1. `compute_image_gradients`
 - Accept one finite, two-dimensional floating-point grayscale array. Return `float32` even when the input uses another floating dtype.
 - Compute

$$I_x[y, x] = \frac{I[y, x+1] - I[y, x-1]}{2}, \quad I_y[y, x] = \frac{I[y+1, x] - I[y-1, x]}{2}.$$

- Positive I_y points downward.
- Set I_x to zero in the first and last columns and I_y to zero in the first and last rows, where the corresponding centered difference is undefined.
- Return aligned floating arrays of shape `(H, W)`.

2. `compute_second_moment`

- Require aligned, finite, two-dimensional gradient arrays and finite `sigma > 0`.
- Form I_x^2 , $I_x I_y$, and I_y^2 .
- Aggregate each product with the supplied normalized separable Gaussian whose radius is $\lceil 3\sigma \rceil$.
- Use reflect-101 padding, which reflects samples without repeating the edge pixel.
- Preserve shape. The notebook default is `sigma=1.5`.

3. `compute_corner_response`

- Require three aligned, finite, two-dimensional moment arrays and a finite scalar $0 \leq k < 0.25$.
- Compute, without clipping or normalization,

$$R = S_{xx}S_{yy} - S_{xy}^2 - k(S_{xx} + S_{yy})^2.$$

- Preserve shape. The notebook default is `k=0.04`.

4. `nonmax_suppression`

- Require a two-dimensional response, $0 \leq \text{threshold} \leq 1$, an integer `radius >= 0`, and an integer `max_points >= 0`.
- Treat nonfinite values as invalid. Let `Rmax` be the largest positive finite response. If no such value exists, or if `max_points == 0`, return empty outputs.
- Retain candidates whose response is at least `threshold * Rmax`. Border-eligible coordinates satisfy `radius <= x < W - radius` and `radius <= y < H - radius`.
- Sort candidates by `(-score, y, x)`.
- Visit candidates in that order. Accept a candidate only when its Euclidean distance from every already accepted point is strictly greater than `radius`; equivalently, an accepted point suppresses candidates at distance `<= radius`.
- Stop at `max_points`. Return keypoints in `(x, y)` order as `int64` shape `(N, 2)` and scores as floating shape `(N,)`, already sorted in acceptance order.
- The notebook defaults are `threshold=0.01`, `radius=8`, and `max_points=500`.

The required B2 figures, listed by input image, are exactly what the two supplied B2 notebook cells produce:

- `square.png` (synthetic square): I_x , I_y , the corner-response image, and the selected points overlaid on the image — four corner clusters before NMS and exactly one final point per corner with the released settings;
- `flat.png` (constant image): the printed maximum response and detection count, both exactly zero — printed evidence suffices, no figure needed; and
- `photo_texture.png` (supplied photograph): selected keypoints at NMS radius 4 and radius 16 side by side, with a short explanation of the density/coverage tradeoff.

No other input image requires B2 evidence.

7.3 B3 — A compact gradient descriptor (10 points)

Implement:

```
describe_keypoints(Ix, Iy, keypoints, patch_size, cells, bins) -> descriptors
```

Use this exact descriptor:

- defaults: `patch_size=16`, `cells=4`, `bins=8`, so $D = 4^2 \cdot 8 = 128$;
- require an even positive `patch_size`, positive integer `cells` and `bins`, and `patch_size` divisible by `cells`;
- require aligned finite gradient arrays and integer-dtype keypoints of shape $(N, 2)$ in (x, y) order; every keypoint center must lie inside the image;
- for keypoint (x, y) , extract bounds $[y-8:y+8, x-8:x+8]$ under the defaults; samples outside the image have zero gradient;
- compute magnitude $m = \sqrt{I_x^2 + I_y^2}$ and orientation $\theta = \text{atan2}(I_y, I_x) \bmod 2\pi$;
- divide the patch into a row-major `cells` x `cells` grid;
- assign each pixel to one hard orientation bin using $\lfloor \theta \text{ bins} / (2\pi) \rfloor \bmod \text{bins}$, weighted by its magnitude;
- flatten cells in row-major order with orientation bin as the fastest-changing index; and
- normalize each vector as $\mathbf{d} / (\|\mathbf{d}\|_2 + 10^{-8})$. A zero patch remains the zero vector.

Do not use spatial/orientation interpolation, Gaussian descriptor weighting, clipping, dominant-orientation alignment, or any hidden library descriptor. Empty keypoints return shape $(0, D)$.

For two selected keypoints, include:

1. the image patch with cell grid and keypoint center;
2. its gradient-magnitude image with a sparse orientation-quiver overlay; and
3. the orientation histogram for every cell, annotated with the dominant cells/orientations.

For one of those keypoints, also show an original/perturbed descriptor comparison and explain what changed. For the perturbation, use one supplied helper from `common/pal_utils.py` — `apply_gaussian_blur`, `add_brightness`, `add_gaussian_noise`, or `rotate_and_scale` — and report the parameter value you chose. Blur, brightness, and noise leave pixel coordinates unchanged, so recompute the descriptor at the *same* keypoint location in the perturbed image; `rotate_and_scale` additionally returns the 2×3 transform it applied, so map your keypoint through that transform to obtain the corresponding location. Do not hand-select a new keypoint in the perturbed image. Report descriptor dimension and L2 norm; do not print all 128 values.

7.4 B4 — Feature matching (8 points)

Implement:

```
match_descriptors(desc1, desc2, ratio) -> (matches, confidence)
```

The first matrix is the reference set and the second is the destination/current set. Use this fixed contract:

- require two floating matrices with equal descriptor width and $0 < \text{ratio} < 1$; the notebook default is `ratio=0.80`;
- discard any descriptor row that is nonfinite or whose L2 norm is at most $1e-8$;
- if either input has no valid rows, or `desc2` has fewer than two valid rows, return `matches` with integer shape $(0, 2)$ and `confidence` with floating shape $(0,)$;
- for every valid row in `desc1`, compute Euclidean distances to valid rows in `desc2` and identify the nearest distance d_1 and second-nearest distance d_2 , breaking equal-distance neighbor ordering by original `desc2` index;
- reject a row when $d_2 \leq 10^{-8}$; otherwise accept it only when the strict test $d_1/d_2 < \text{ratio}$ passes;
- store each accepted pair as original input indices `[index1, index2]` and compute

$$\text{confidence} = 1 - \frac{d_1}{d_2 + 10^{-8}};$$

- sort accepted pairs by `(-confidence, index1, index2)`.

This is a one-way ratio test, so different reference descriptors may select the same destination descriptor.

Use the supplied easy pair for the required match plot. Diagnose one convincing match and one incorrect or ambiguous correspondence using appearance, distance, ratio, and/or geometry. If every accepted match is correct, diagnose one rejected ambiguous candidate instead of inventing an error.

8. Part C — UNDERSTAND: Stress-Test the Representation (15 points; 1.5–2 hours)

Generative AI is permitted with disclosure. It may help organize experiments or plots, but your conclusions must be supported by your own recorded evidence.

Work order. Read all of Part C before running anything. C2 asks for predictions, so write them down *before* you run the C1 experiments. C3 asks for a tracking failure; it is expected (and easiest) to complete C3 after building Part D and report the result here.

8.1 C1 — Four invariance experiments (8 points)

Run exactly these four conditions against one shared reference view that you capture yourself (or use under the approved capture alternative in Section 10):

1. a real illumination change captured with the camera;
2. a real physical scale and/or viewpoint change captured with the camera;
3. synthetic Gaussian blur; and
4. synthetic in-plane rotation.

Use at least one clearly nontrivial severity for each condition and record the physical setup or synthetic parameter. Use the supplied transforms for synthetic conditions so their parameters are reproducible. An approved course capture alternative may replace the camera clips only under the policy in Section 10.

For every condition, report the same quantities:

- number of reference features;
- number of destination features;
- number of ratio-test-accepted matches;
- number of robust-estimator inliers; and
- inlier fraction, defined as `inliers / accepted matches` and reported as zero when no match is accepted.

Put all four rows in one compact table. Also show a readable match/inlier visualization for each condition and give a one- or two-sentence result summary.

C1 protocol — read before capturing.

- *Stills, not clips.* Every C1 condition compares still images: capture one reference photo, re-photograph the same scene for conditions 1–2, and apply the supplied transforms to the reference photo itself for conditions 3–4. No video is used in C1.
- *Reusing Part D material is allowed and encouraged:* your Part D target may serve as the C1 target, and the reference view may be a sharp photo of your Part D setup (a clean extracted frame is also acceptable).
- *Feature region.* Detect, describe, and match over the full image in both views. Do not restrict detection to a target ROI in Part C; the ROI machinery belongs to Part D.
- *Fixed parameters.* Every row uses the released defaults: `sigma=1.5`, `k=0.04`, NMS threshold `0.01` and radius 8, at most 500 points, the 16/4/8 descriptor, `ratio=0.80`, and the supplied estimator defaults. Do not tune parameters per condition; parameter studies belong in C2 discussion or Part E, clearly labeled as extra.
- *Suggested severities* (anything in these ranges counts as clearly nontrivial; record the value or setup you used):
illumination — an obviously different light source, lamp position, or strong shadow, not a slight dimming;
scale/viewpoint — roughly $1.5\text{--}2\times$ distance change or a $20\text{--}40^\circ$ viewpoint change; blur — $\sigma \in [1.5, 3]$; rotation — $15\text{--}45^\circ$.
- *Exactly four rows.* One severity per condition. You may run additional severities and discuss them in C2 or Part E, but the required C1 table contains exactly these four rows.
- *Inliers are not ground truth.* Estimator inliers measure agreement with one fitted geometric model; they are **not** independently verified correct matches. A repetitive scene can yield consistent-but-wrong inliers, and a correct match can land outside the consensus set.

Capture checklist: same target and framing in all views; camera steady or braced; exposure locked (or noted) if your camera allows it; reference view sharp and well lit; originals kept at full resolution.

Table template:

Condition	Ref. feats	Dest. feats	Matches	Inliers	Inlier frac.
Illumination (setup: ...)					

Condition	Ref. feats	Dest. feats	Matches	Inliers	Inlier frac.
Scale/viewpoint (setup: ...)					
Blur ($\sigma = \dots$)					
Rotation ($\theta = \dots^\circ$)					

8.2 C2 — Connect experiment to theory (4 points)

Choose two C1 conditions. For each:

1. record what you predicted before running it;
2. state what happened, citing the C1 measurements; and
3. explain the result with a specific idea from Part A or Part B.

At least one of the two explanations must analyze descriptor invariance. Label prediction and observation separately so hindsight does not replace a prediction.

8.3 C3 — Failure case (3 points)

Find one case where the tracker clearly fails. Show the failed result and diagnose the most likely cause with pipeline evidence. Possible causes include insufficient texture, repetition, severe blur, large viewpoint change, occlusion, too few points, ambiguous descriptors, or an invalid transform. The objective is diagnosis, not a universal fix.

9. Part D — CREATE: Object-Locked Video (25 points; 2.5–3 hours)

Generative AI is permitted for Part D with disclosure, but you remain responsible for every submitted behavior. The supplied geometry is a black box for this assignment; the student-built Part B features are not.

9.1 D1 — Capture and target selection (4 points)

Record a 10–20 second landscape handheld clip, preferably 720p at 30 fps. Select a textured, matte, approximately rigid target that occupies about 25–60% of the first frame. Keep it visible while introducing moderate translation, rotation, scale change, and limited viewpoint change. Avoid rapid motion, severe blur, glossy or featureless surfaces, and highly repetitive patterns.

In the first frame, draw one target bounding box. Reference keypoint centers must lie inside this region. As in the supplied harness, detect over the full first frame, then retain the centers inside the box; their descriptor patches may extend beyond the box. The box uses processed-frame coordinates (x, y , width, height). Include the boxed frame, capture settings, target description, and motion description in the notebook and PDF.

9.2 D2 — Track and stabilize (8 points)

Build the video pipeline with these required choices:

1. Detect and describe the first frame once, then retain only reference keypoint centers inside the target box.
2. Detect and describe each current frame over the full processed frame.
3. Match reference descriptors (`desc1`) directly to current descriptors (`desc2`) on every frame. Do not chain descriptors or transforms frame to frame.
4. Form `ref_pts = ref_keypoints[matches[:, 0]]` and `cur_pts = cur_keypoints[matches[:, 1]]`.
5. Pass `cur_pts` as source and `ref_pts` as destination to the supplied similarity-transform estimator, so the estimated mapping is current-to-reference.
6. Use the supplied `cv2.estimateAffinePartial2D` RANSAC wrapper with a 3-pixel threshold at processed width at most 640 pixels, 2000 iterations, and confidence `0.99`.
7. Require at least four accepted matches and at least three inliers before accepting a transform.
8. On estimator failure, hold the most recent valid transform for at most five consecutive frames. On the sixth and later failed frames, emit an unwarped frame visibly marked as tracking failure until a valid transform is reacquired.
9. Use only the supplied mild parameter exponential moving average to reduce estimator jitter; do not accumulate frame-to-frame transforms.
10. Process at width at most 640, 10–15 sampled frames per second, and at most 500 keypoints per frame. The supplied reader samples the nearest source frames at the requested output timestamps and returns the actual output rate. Encode with that returned rate so duration remains correct. If your source is below 10 fps, recapture at a higher frame rate or ask staff about the approved alternative.

Two boundary behaviors are the supplied harness's responsibility, not yours. Before the first accepted transform, held frames fall back to the identity transform — they display unwarped but are labeled as held, under the same five-frame limit. When tracking resumes after a failure run, the new estimate is blended with the held transform by the same exponential smoothing (weight 0.8 on the new estimate), so the lock eases back in over a few frames rather than snapping. Every output frame is labeled with its state (`ok`, `held_last_transform`, or `unwarped_after_failure`), and the same state appears in the returned per-frame diagnostics.

Plot or tabulate frame-level match count, inlier count, and failure state. Briefly explain one design choice or debugging change. Your calls to the supplied estimator, warper, and encoder may be AI-assisted; your B2–B4 functions may not be.

9.3 D3 — Hero result (10 points)

Produce one polished, self-contained 10–20 second video that makes the object-locked effect immediately understandable. The video must:

- show a labeled original/stabilized comparison, either side by side or as a clearly timed before/after presentation;
- keep the selected target approximately fixed while visible surrounding content moves;
- use your own capture or a staff-approved course alternative;
- remain understandable without opening the notebook; and
- be an H.264 MP4 with yuv420p pixel format, at most 1280x720 and 30 fps, and at most 50 MB.

Titles, captions, music, and simple editing are optional. They do not replace technical evidence, target lock, or playability.

9.4 D4 — Comprehension reflection (3 points)

Write 150–250 words and use one of the following paths.

If you used generative AI in a permitted section:

1. What did it help you implement or produce?
2. How does the most important AI-assisted portion work?
3. What did you change, debug, test, or redesign from the initially generated version?

If you did not use generative AI:

1. State that no generative AI was used.
2. Explain how the most important Part D portion works.
3. Describe a concrete change, debugging step, or test that improved the result.

10. Capture access, privacy, safety, and accessibility

Do not record non-consenting people, private documents, personal information on screens, or unsafe/prohibited locations. Do not take risks to obtain a more dramatic result.

A course-provided capture alternative is available only with staff approval for access, privacy, safety, or accessibility reasons. Request approval through the course staff help channel or office hours before relying on it. Approved students receive a staff capture pack: a reference photograph of a compliant textured target, re-photographed illumination-change and scale/viewpoint-change views of the same target (completing C1 conditions 1–2), and one compliant 10–20 second handheld source video with its first-frame target box (for Part D). With the pack, every part is completed identically — students still apply the two synthetic C1 conditions to the pack’s reference photo, still select the target box, and still run the full pipeline themselves — and grading is identical.

11. Part E — EXPLORE: Optional extension (up to +10 points; about 1 hour)

Generative AI is permitted with disclosure. Extend, improve, compare, or deliberately break the method in a way not already required. Examples include recovery after occlusion, multiple targets, comparison with SIFT or a learned matcher, a more invariant descriptor, improved stabilization, or an adversarial scene.

- **+5:** a meaningful additional exploration with a clear result and concise technical explanation.
- **+10:** a particularly thoughtful, technically interesting, creative, or ambitious exploration with convincing evidence.

There are no undocumented intermediate levels. Part E and showcase participation share one **10-point total extra-credit cap**.

Public showcase participation is optional. Students who opt in may post the final artifact to the designated course Slack channel within 24 hours after the assignment deadline. Voting remains open for 72 hours using the designated `:movie_camera:` reaction. Participation earns +1 within the shared cap; vote count never determines academic credit. The course may recognize a People’s Choice result and a non-credit Technical Highlight. Public posting requires the student’s affirmative choice.

12. Submission specification

Submit through Gradescope by **September 29, 2026, 11:59 pm ET**.

Submit:

1. `pal.py` with the required implementations;
2. the completed `pal_starter.ipynb`, with required evidence outputs retained;
3. one LaTeX-generated written PDF made from `answer_template.tex`;
4. one compliant hero-result MP4; and
5. collaborator, outside-resource, and generative-AI acknowledgments.

Save figures under `artifacts/` and insert them into the provided LaTeX template; the notebook includes save/download examples. Back up edited code and captures before disconnecting a Colab runtime.

Every required figure and table must appear in both the executable notebook and the corresponding PDF section. The hero MP4's labeled original/object-locked comparison supplies source-capture evidence; do not upload a separate raw capture unless staff requests it. Retain raw data locally until grades are final.

Use the stable A1–D4 labels in the provided template. Begin each labeled subproblem on a new page using the page breaks already included in the template, in template order. No answer has a fixed page number: when an answer runs long, it simply pushes every later answer to a later page, which is expected and fine. After upload, assign *every* PDF page containing part of an answer to that question in Gradescope — a question may span several pages and a page may hold several questions — and inspect the rendered preview before finishing. Do not insert unrelated cover or scratch pages.

The compiled written PDF is uploaded to the Gradescope assignment **PA1 — Written**; `pal.py`, the executed notebook, and the hero MP4 are uploaded to **PA1 — Code**. The acknowledgments-and-disclosure section is the final labeled section of the answer template and is submitted inside the written PDF; there is no separate form.

The final video must be H.264 MP4, yuv420p, 10–20 seconds, at most 1280×720 and 30 fps, and at most 50 MB. All required code must execute in a fresh Google Colab runtime without paid compute, credentials, or private services.

13. Point summary

Part	Points
A1	10
A2	7
A3	4
A4	4
B1	5
B2	12
B3	10
B4	8
C1	8
C2	4
C3	3
D1	4
D2	8
D3	10
D4	3
Base total	100
E and showcase combined	0 to +10